

Universidad Autónoma de Madrid
Escuela Politécnica Superior



PROYECTO: APRENDIZAJE AUTOMÁTICO III

Grado en Ciencia e Ingeniería de Datos.

Autor: Iván Domínguez Hernández

Autor: Lucas Miranda López

Tutor: Alberto Suárez González

30 de abril de 2026

MODELOS DE DIFUSIÓN PARA IA GENERATIVA

Autor: Iván Domínguez Hernández

Autor: Lucas Miranda López

Tutor: Alberto Suárez González

Departamento de Aprendizaje Automático
Escuela Politécnica Superior
Universidad Autónoma de Madrid

30 de abril de 2026

Abstract/Resumen

TODO: Resumen en inglés, 250-500 palabras.
TODO: Resumen en español, 250-500 palabras.

Índice general

1. Introducción	1
1.1. Alcance	1
1.2. Estructura del documento	1
2. Estado del Arte	2
2.1. Introducción al problema	2
2.1.1. Objetivo	2
2.1.2. Estimación de densidad	2
2.2. Modelos de difusión	3
2.2.1. Proceso Forward (inyección de ruido)	3
2.2.2. Desarrollo de la estimación de $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ y la función de pérdida	4
2.2.3. Proceso backward (generación)	5
2.2.4. Variance Exploding (VE)	6
2.2.5. Variance Preserving (VP)	7
2.2.6. Ecuación de Fokker-Planck	8
2.3. Modelos de procesos de difusión para generación condicionada	9
2.3.1. El problema de la generación condicionada	9
2.3.2. Classifier guidance	10
2.3.3. Classifier-free guidance	10
2.3.4. Función de pérdida	11
2.3.5. Planteamiento de las SDEs	11
2.3.6. SDE backward condicionada	11
2.3.7. Imputación	12
2.4. Fenómeno del double descent	12
3. Desarrollo de Software y Diseño	15
3.1. Análisis de Requisitos	15
3.1.1. Requisitos funcionales	15
3.1.2. Requisitos no funcionales	16
3.2. Casos de Uso	17
3.3. Arquitectura y Diseño	18
3.3.1. Estructura del paquete	18
3.4. Validación y Calidad del Software	18
3.4.1. Metodología de desarrollo	18
3.4.2. Guías de estilo	18
3.4.3. Tests automatizados	19

4. Desarrollo del proyecto y resultados	20
4.1. VE vs VP	20
4.2. Schedule lineal vs Schedule coseno en VP	20
4.3. VE / VP vs Fokker Planck	20
4.4. Generación condicionada	20
4.5. Medición de métricas en test para los checkpoints de los gatos.	20
5. Conclusiones	21
Bibliografía	I

Capítulo 1

Introducción

1.1. Alcance

El objetivo de este proyecto es construir un sistema de Inteligencia Artificial generativa altamente configurable, capaz de sintetizar imágenes monocromáticas y a color. Para lograrlo, el alcance técnico abarca la implementación de los siguientes componentes:

- **Procesos de difusión:** Modelos *Variance Exploding* (VE) y *Variance Preserving* (VP).
- **Muestreadores (*Samplers*):** Integrador Euler-Maruyama, esquemas Predictor-Corrector y ecuaciones diferenciales ordinarias de flujo de probabilidad (*Probability flow ODE*).
- **Programación de ruido (*Noise schedules*):** Planificaciones de decaimiento lineal y tipo coseno.
- **Métricas de evaluación:** *Bits Per Dimension* (BPD), *Fréchet Inception Distance* (FID) e *Inception Score* (IS).
- **Generación controlable:** Muestreo condicionado a clases y técnicas de imputación.

1.2. Estructura del documento

Para facilitar su lectura técnica, el informe se organiza de la siguiente manera: el **Capítulo 2** expone el Estado del Arte, marco teórico y formulación matemática de los modelos. El **Capítulo 3** aborda el Diseño del Software, detallando la arquitectura, casos de uso y planificación temporal. El **Capítulo 4** presenta el Desarrollo y los Resultados, evaluando el rendimiento y la calidad gráfica del sistema. Finalmente, el **Capítulo 5** recoge las Conclusiones extraídas, cerrando el documento con la bibliografía y los apéndices técnicos correspondientes.

Capítulo 2

Estado del Arte

2.1. Introducción al problema

Un modelo generativo es un modelo de probabilidad conjunta $p_0(\mathbf{x})$, $\mathbf{x} \in \mathcal{X}$ que puede estar condicionado por inputs o clases condicionales $c \in \mathcal{C}$, dando lugar a un modelo generativo condicional $p_0(\mathbf{x}|c)$. Existen muchos modelos generativos, entre los que se encuentran los DGM (Deep Generative Models) y los PGM (Probabilistic Graphical Models) con un enfoque más clásico. No son necesariamente excluyentes y por supuesto pueden existir modelos generativos híbridos. Los principales DGM son los VAE (Variance Autoencoders), los ARM (Autoregressive Models), difusión y EGM (Energy Based Models) o los GAN (Generative Adversarial Networks) [1].

2.1.1. Objetivo

El principal objetivo del proyecto es generar nuevas muestras de datos, concretamente imágenes, un problema que en general es conocido como IA generativa.

2.1.2. Estimación de densidad

La estimación de densidad se presenta formalmente como el problema subyacente a evaluar la probabilidad de observar un vector de datos concreto, como en nuestro caso será la $p(\mathbf{x})$, introducida anteriormente. Una aproximación simple a este problema es el Kernel Density Estimation (KDE), que tiene la forma:

$$p(\mathbf{x}|\mathcal{D}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^D} K\left(\frac{\|\mathbf{x} - x_n\|_2^2}{h^2}\right)$$

Donde los datos $\mathcal{D} = \{x_n\}_{n=1}^N$, $x_n \in \mathbb{R}^D$, K es una función de densidad de kernel tal que $K : \mathbb{R} \rightarrow \mathbb{R}^+$ y tal que $\int K(\mathbf{x})d\mathbf{x} = 1$ y $\int \mathbf{x}K(\mathbf{x})d\mathbf{x} = 0$ y h el paso de banda asociado a una tal estimación de densidad para los datos $\mathcal{D}$ [1]. En tratamiento de imágenes, se piensa en el problema como uno de tantas dimensiones como píxeles tenga. Como se presenta en [2, Section 3.1], en espacios con una dimensionalidad tan alta, se puede demostrar que la masa de distribución gaussiana $\mathcal{N}(0, \mathbb{I}_d)$ se concentra en la superficie de una esfera de radio $r \approx \sqrt{d}$, lo que hace que, en general, las distancias entre pares de muestras sean parecidos y que, por tanto, el efecto del kernel se disipe. Dicho de otra forma, las imágenes viven en $\mathbb{R}^D$, pero no llenan el espacio, viven en una variedad dimensional mucho menor. Es en este contexto donde los modelos profundos cobran relevancia: su capacidad para aprender representaciones latentes de baja dimensión les permite capturar la estructura intrínseca de los datos sin necesidad de modelar $\mathbb{R}^D$ directamente.

2.2. Modelos de difusión

2.2.1. Proceso Forward (inyección de ruido)

Los modelos de difusión serán aquellos capaces de estimar $p(\mathbf{x})$, siendo p la función de probabilidad desconocida asociada precisamente al dato real $\mathbf{x}_0 \in \mathbb{R}^D$, $\mathbf{x}_0 \sim p(\mathbf{x})$ siguiendo un procedimiento basado en la inyección de ruido gaussiano en T pasos, produciendo una secuencia de muestras ruidosas:

$$\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$$

Donde el tamaño de los pasos está controlado por un cronograma de varianza que no definiremos todavía. La idea de inyectar ruido en las imágenes es conseguir que un modelo aprenda donde añadir o eliminar ruido para que obtener la imagen $\mathbf{x}_0$ original. De esta forma, $\mathbf{x}_0$ se va difuminando a la par que se mezcla con el ruido. Es fácil notar que cuando $T \rightarrow \infty$, $p(\mathbf{x}_T) \sim \mathcal{N}(0, \sigma^2 \mathbb{I})$, pues llega un momento temporal en que la media del dato original $\mathbb{E}[\mathbf{x}_0]$ queda opacada por ruido tras tantas inserciones del mismo, con lo que se puede decir que $\mathbb{E}[\mathbf{x}_T] \rightarrow 0$ termina siendo una normal isotrópica de varianza desconocida $\sigma^2 \in \mathbb{R}$ [3]. Este resultado no es trivial y se usará más adelante.

La forma genérica de inyección de ruido es usando ecuaciones diferenciales estocásticas. Concretamente su forma genérica, propuesta por Itô:

$$d\mathbf{x} = f(\mathbf{x}(t), t)dt + g(t)dW$$

Por ejemplo, para un caso trivial en el que $f(\mathbf{x}(t), t) = 0$ y $g(t) = 1$, la inyección de ruido sería precisamente el movimiento browniano por definición:

$$d\mathbf{x} = dW$$

La discretización de Euler-Maruyama de este movimiento es trivial:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \sqrt{\Delta t} \mathbf{z}_t$$

El cronograma de varianza se define por el proceso que controla la cantidad de ruido que se añade en cada paso. Para el caso en que el cronograma de varianza venga dado por:

$$\beta = \{\beta_t \in (0, 1)\}_{t=1}^T, \quad \beta_1 < \beta_2 < \dots < \beta_T$$

y los términos f y g por:

$$f(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x}, \quad g(t) = \sqrt{\beta(t)}$$

La ecuación diferencial resultante sería:

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}dW \text{ (forward discreto)}$$

Cuya discretización Euler-Maruyama es:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{1}{2}\beta(t)\mathbf{x}_t\Delta t + \sqrt{\beta(t)\Delta t}\mathbf{z}_t \text{ (forward discreto)}$$

Nota: en ambos casos, el proceso de Wiener se convierte en ruido distribuido normalmente: $\mathbf{z}_t \sim \mathcal{N}(0, \mathbb{I})$.

2.2.2. Desarrollo de la estimación de $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ y la función de pérdida

Un caso particular de la SDE propuesta por Itô recuerda particularmente a un fenómeno de la física, las dinámicas de Langevin, cuyo objetivo inicial era el modelado estadístico de sistemas moleculares. Describe el movimiento de una partícula en un fluido viscoso sometida a fuerzas aleatorias (ruido térmico). Su forma estocástica es:

$$d\mathbf{x} = -\nabla U(\mathbf{x})dt + \sqrt{2}dW$$

donde $-\nabla U(\mathbf{x})$ es una fuerza determinista que representa el gradiente de la energía potencial (apunta hacia mínimos) y dW es movimiento browniano, que perturba aleatoriamente [4]. Fue Aapo Hyvärinen [5] quien introdujo por primera vez la intuición de estimar el gradiente de la log probabilidad de los datos observados $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ con el objetivo de estimar la función de densidad de probabilidad real de los datos $p(\mathbf{x})$. Esto puede interpretarse como la dirección de máximo ascenso de una superficie de probabilidad. Hyvärinen planteó la función de coste:

$$J(\theta) = \int_{\xi \in \mathbb{R}^n} p_{\mathbf{x}}(\xi) \sum_{i=1}^n \left[\partial_i \psi_i(\xi; \theta) + \frac{1}{2} \psi_i(\xi; \theta)^2 \right] d\xi + \text{const}$$

con $\psi_i(\xi; \theta) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$ (idealmente aproximado por un modelo en concreto, de parámetros θ fijos) y $J(\theta)$ (la función objetivo a minimizar). El subíndice i se refiere a la i -ésima componente. La idea de estimar $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ mediante el aprendizaje automático quedó arraigada a la literatura, con el objeto de encontrar un modelo capaz de intuir donde añadir o eliminar ruido de unos ciertos datos para conseguir generar imágenes. La síntesis de la intuición de Hyvärinen se materializó en [6], donde Vincent apuntó a que la función de reconstrucción $r(\mathbf{x})$ de los autoencoders es proporcional al gradiente de la log probabilidad estimada:

$$r(\mathbf{x}) \propto \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

un avance que permitía usar los modelos más flexibles hasta entonces, las redes neuronales, capaces de estimar el score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ como una caja negra, denotando ahora la estimación del modelo por $s_{\theta}(\mathbf{x}, \sigma)$. Para el entrenamiento mediante redes neuronales, Song [7] avanzó una función de pérdida distinta a la de Hyvärinen, más centrada en la detección del ruido:

$$\ell(\theta; \sigma) = \frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{\mathbf{x} \sim \mathcal{N}(\mathbf{x}, \sigma^2 I)} \left[\|\mathbf{s}_{\theta}(\mathbf{x}, t) - \nabla_{\mathbf{x}} \log p(\mathbf{x}, t)\|_2^2 \right]$$

Cuya simplificación se deduce en los apéndices y da como resultado:

$$\ell(\theta; \sigma) = \frac{1}{N} \sum_{n=1}^N \frac{\lambda(t_i)}{\sigma^2(t_i)} \|\sigma^2(t_i) s_{\theta}(\mathbf{x}_n, \sigma) + \mathbf{z}_n\|_2^2, \quad \mathbf{z}_n \sim \mathcal{N}(0, 1)$$

donde $\lambda(t_i)$ se introduce como una función de ponderación que controla el peso de cada nivel de ruido en cada paso. La elección que propone Song [7] es $\lambda(t_i) = \sigma^2(t_i)$, dando lugar a la función de coste definitiva que se usará más adelante:

$$\ell(\theta; \sigma) = \frac{1}{N} \sum_{n=1}^N \|\sigma^2(t_n) s_{\theta}(\mathbf{x}_n, \sigma) + \mathbf{z}_n\|_2^2, \quad \mathbf{z}_n \sim \mathcal{N}(0, 1) \quad (2.1)$$

Una vez está claro como inyectar el ruido, y la forma de la función de pérdida, puede entenderse como la red neuronal realiza una estimación $s_0(\mathbf{x}, \sigma)$. Como se desarrollará en la sección 4, la red resolverá un problema de regresión entre las imágenes ruidosas y el tiempo de ruido que se las ha inyectado.

2.2.3. Proceso backward (generación)

Anderson (1982) [8] demostró que la SDE inversa exacta es:

$$d\mathbf{x} = [f(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p(\mathbf{x})]dt + g(t)dW$$

Recordatorio: como se introdujo anteriormente, $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ es la dirección (que se aproximará por redes neuronales) de máximo crecimiento a la distribución real de los datos, que en este documento se denota por $p(\mathbf{x})$. Por ejemplo, para el caso trivial que veíamos antes en el que $f(x, t) = 0$ y $g(t) = 1$:

$$d\mathbf{x} = \nabla_x \log p(\mathbf{x})dt + dW$$

Como se vio antes, este caso define la dinámica de Langevin. El objetivo de la siguiente subsección es introducir cómo se llega a la función de pérdida para entrenar a la U-Net, así como desarrollar el proceso forward (inyección de ruido) y backward (proceso generativo) de Langevin. Posteriormente, se desarrollará, de una forma más sencilla, para un caso más complejo, la SDE de Ornstein-Uhlenbeck, cuya expresión viene determinada por:

$$f(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x}, \quad g(t) = \sqrt{\beta(t)}$$

En cuyo caso:

$$d\mathbf{x} = [-\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)\nabla_{\mathbf{x}} \log p(\mathbf{x})]dt + \sqrt{\beta(t)}dW$$

Langevin

Song, además, propone por primera vez la arquitectura de la U-Net para estimar $s_{\theta}(\mathbf{x}, \sigma)$, así como la primera propuesta formal de usar dinámicas de Langevin que mencionábamos al principio para generar nuevos ejemplos:

$$d\mathbf{x} = \sigma \nabla_{\mathbf{x}} \log p(\mathbf{x})dt + \sqrt{\sigma}dW \quad (\text{backward})$$

donde en lugar de fijar $f(\mathbf{x}, t) = 0$, $g(t) = 1$, fijaba $f(\mathbf{x}, t) = \sigma$, $g(t) = \sqrt{\sigma}$. Esta ecuación diferencial estocástica (SDE) tiene discretización de Euler-Maruyama de la forma:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \sigma \nabla_{\mathbf{x}} \log p(\mathbf{x})\Delta t + \sqrt{\sigma}\mathbf{z}_t \quad (\text{backward discreto})$$

donde, siguiendo la convención que se definió en secciones anteriores, $\sigma \nabla_{\mathbf{x}} \log p(\mathbf{x})$ es el término de deriva y $\sqrt{\sigma}$ el de difusión.

Ornstein-Uhlenbeck

Si en lugar de usar la ecuación diferencial estocástica de Langevin usáramos la de Ornstein-Uhlenbeck:

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)}dW(t)$$

Su inversa sería:

$$d\mathbf{x} = [-\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)\nabla_{\mathbf{x}} \log p(\mathbf{x})]dt + \sqrt{\beta(t)}dW$$

Cuya discretización Euler-Maruyama es:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \left[\frac{1}{2}\beta(t)\mathbf{x}_t + \beta(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] \Delta t + \sqrt{\beta(t)\Delta t}\mathbf{z}_t \quad (\text{backward discreto})$$

2.2.4. Variance Exploding (VE)

En 2021, Song [9] propone un modelo que bautiza como Variance Exploding. Tiene como condiciones

$$f(\mathbf{x}, t) = 0 \text{ y un } g(t) = \sqrt{\frac{\partial[\sigma^2(t)]}{\partial t}}, \quad t \in [0, 1]$$

La idea de este modelo es que, al no tener término de drift, necesita perturbar los datos con varianzas altas. En estos casos, el score es más fácil de estimar, puesto que las imágenes perturbadas se alejan de la original. Pero el hecho de que la varianza explote no es una ventaja en sí mismo, es la consecuencia de no tener drift, no es un objetivo de diseño [9].

Con la definición anterior podemos determinar sencillamente las ecuaciones de forward y backward discretizadas por Euler-Maruyama de igual forma que hicimos en secciones anteriores. Su SDE es:

$$d\mathbf{x} = g(t)dW \quad (2.2)$$

cuya versión discreta sería:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + g(t)\mathbf{z}_t \quad (2.3)$$

Su backward sería:

$$d\mathbf{x} = -g(t)^2 \nabla_{\mathbf{x}} \log p(\mathbf{x}) dt + g(t)dW \quad (2.4)$$

con versión discreta:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + g(t)^2 \nabla_{\mathbf{x}} \log p(\mathbf{x}_t) \Delta t + g(t) \sqrt{\Delta t} \mathbf{z}_t \quad (2.5)$$

Ambas con $\mathbf{z}_t \sim \mathcal{N}(0, \mathbb{I})$ e iterando para $t = \Delta t, 2\Delta t, \dots, T$ en la versión forward y $t = T, T - \Delta t, T - 2\Delta t, \dots, \Delta t$ para la backward. Ahora bien, $\sigma^2(t)$ no se define hasta el momento de ninguna forma en particular. Teóricamente podría definirse de diversas maneras. Pero eso es lo que diferencia a Variance Exploding de otros modelos generativos. Song propone usar un estándar tal que:

$$\sigma(t) = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^t \approx \sigma^t$$

Con un σ que se calcula de forma heurística (al menos como propone Song [9]) y depende de las imágenes con las que se va a entrenar el modelo (por ejemplo, para los dígitos de MNIST la literatura propone usar $\sigma = 25$). Aunque otra propuesta de Song en ese mismo artículo es elegirlos de forma que $\sigma_{\min}$ sea suficientemente pequeño para perturbar los datos y $\sigma_{\max}$ sea del orden de la distancia máxima entre pares de muestras del dataset. De esta forma que cuando $T \rightarrow 1$, la varianza explote. La inserción de ruido se realiza de forma desproporcionada y abrupta cuando T se encuentra en los alrededores de 1. Así, podemos determinar las SDE que se implementarán en el código del proyecto en la siguientes secciones:

$$g(t) = \sqrt{\frac{\partial[\sigma^2(t)]}{\partial t}} = \sqrt{\frac{\partial[\sigma^{2t}]}{\partial t}} = \sqrt{2\sigma^{2t}} \propto \sigma^t$$

Y por lo tanto, las versiones discretas de forward y backward que se plantearán más adelante en el proyecto serán, definitivamente:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \sigma^t \mathbf{z}_t \quad (\text{Forward VE}) \quad (2.6)$$

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \sigma^{2t} \nabla_{\mathbf{x}} \log p(\mathbf{x}_t) \Delta t + \sigma^t \sqrt{\Delta t} \mathbf{z}_t \quad (\text{Backward VE}) \quad (2.7)$$

Con las mismas $\mathbf{z}_t$ y t que antes para las versiones forward y backward.

2.2.5. Variance Preserving (VP)

Lo común es basar los modelos de Variance Preserving en los procesos de Ornstein-Uhlenbeck. En este método el término de drift no se hace 0, por lo que hay diferentes propuestas para la inyección de ruido, no tan solo una en la que explota como en VE. Recordemos que la SDE de Ornstein-Uhlenbeck es de la forma:

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)}dW(t) \quad (2.8)$$

Y su backward sería:

$$d\mathbf{x} = \left[-\frac{1}{2}\beta(t)\mathbf{x}_t + \beta(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] dt + \sqrt{\beta(t)}dW \quad (2.9)$$

Por tanto su versión discretizada por Euler-Maruyama:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \frac{1}{2}\beta(t)\mathbf{x}_t\Delta t + \sqrt{\beta(t)}\mathbf{z}_t \quad (\text{Forward VP}) \quad (2.10)$$

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \left[\frac{1}{2}\beta(t)\mathbf{x}_t + \beta(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] \Delta t + \sqrt{\beta(t)\Delta t}\mathbf{z}_t \quad (\text{Backward VP}) \quad (2.11)$$

Ambas con $\mathbf{z}_t \sim \mathcal{N}(0, 1)$. Song, en 2021 [9], propuso un schedule de varianza $\{\beta_t\}_{t=0}^1$ lineal. Esta idea concuerda con el hecho de que no eliminamos el término de drift. Este schedule propuesto es, concretamente:

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$$

con $\beta_{\min} = 0,01$, $\beta_{\max} = 20$ y $t \in [0, 1]$. Pero como explica Nasir en [10], este schedule lineal no es adecuado para imágenes de baja resolución, pues las muestras se vuelven demasiado ruidosas demasiado temprano, impidiendo que la red neuronal aprenda adecuadamente. Es por eso que, como desarrolla Hang [11], es más útil usar un schedule de tipo coseno en estos casos. Define:

$$\alpha_t = \prod_{s=1}^t (1 - \beta_s)$$

Con:

$$\alpha_t = \frac{f(t)}{f(0)}, \text{ donde } f(t) = \cos\left(\frac{\frac{t}{T} + s}{1 + s} \times \frac{\pi}{2}\right)$$

De nuevo, se fijan los hiperparámetros. Concretamente $s = 0,008$ (para evitar que haya muy poco ruido al principio) y las $\beta(t)$ se recuperan de la forma:

$$\beta(t) = 1 - \frac{\alpha_t}{\alpha_{t-1}}$$

Un ejemplo para la diferencia entre ambos schedules (linear arriba, coseno abajo) para una imagen a la que se le inyecta ruido en pasos $t \in [0, T]$ lo presentan Nichol & Dhariwal (2021):

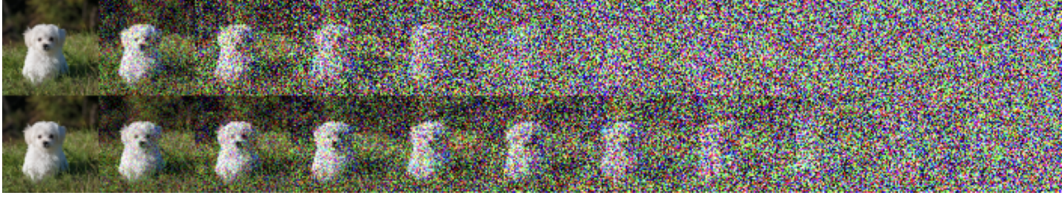


Figura 2.1: Nichol & Dhariwal (2021), [12].

En este artículo también presentan la ventaja en términos de la negative log-likelihood, que mejora para este schedule sin empeorar FID (métricas que se verán más adelante, capaces de medir la calidad de las imágenes).

2.2.6. Ecuación de Fokker-Planck

Sea $X_t \in \mathbb{R}^{\mathcal{D}}$ un proceso estocástico gobernado por la SDE de Itô

$$dX_t = f(X_t, t) dt + g(t) dW_t,$$

donde $f : \mathbb{R}^{\mathcal{D}} \times [0, T] \rightarrow \mathbb{R}^{\mathcal{D}}$ es el coeficiente de deriva, $g : [0, T] \rightarrow \mathbb{R}$ es un coeficiente de difusión escalar y W_t es un proceso de Wiener estándar $\mathcal{D}$ -dimensional. En lugar de seguir trayectorias individuales, la ecuación de Fokker-Planck (también conocida como ecuación de Kolmogorov hacia adelante) describe cómo evoluciona en el tiempo la densidad de probabilidad $p_t(x) = p(x, t)$ de X_t [13, 14]:

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot [f(x, t) p_t] + \frac{g(t)^2}{2} \Delta p_t$$

El primer término se denomina operador de transporte: la deriva f desplaza la masa de probabilidad a través del espacio de estados. El segundo término es un operador de difusión: el ruido dispersa dicha masa. La ecuación anterior se obtiene aplicando la fórmula de Itô a una función test arbitraria $\phi \in \mathbb{R}^{\mathcal{D}}$, tomando esperanzas e integrando por partes [15].

La idea central de los modelos de difusión basados en el *score* [9] consiste en explotar la ecuación de Fokker-Planck en dos etapas. En primer lugar, se elige una SDE de la forma SDE forward general de Itô, de modo que p_t evolucione desde una distribución de datos arbitraria p_0 , habitualmente $\mathcal{N}(0, I)$, conforme $t \rightarrow T$. En segundo lugar, Anderson [8] demostró que toda difusión cuya densidad p_t satisface la ecuación de Fokker-Planck admite una SDE en tiempo inverso cuyos marginales en el instante $T - t$ coinciden exactamente con p_t . Generar muestras de p_0 se reduce entonces a (i) muestrear X_T de la distribución a priori y (ii) integrar la SDE inversa desde $t = T$ hasta $t = 0$. Siguiendo el esquema de VP de [16], el proceso *forward* viene dado por la ecuación (2.7), donde $\beta : [0, T] \rightarrow \mathbb{R}_+$ es el schedule de ruido. Como la SDE forward es lineal en X_t , el núcleo de transición es gaussiano y admite forma cerrada:

$$X_t = \alpha_t X_0 + \sigma_t \epsilon$$

con $\epsilon \sim \mathcal{N}(0, I)$, lo que permite obtener cualquier muestra ruidosa en un único paso sin simular la trayectoria completa. La discretización de Euler-Maruyama de la SDE forward particionando $[0, T]$ en pasos de tamaño Δt obtenemos:

$$X_{t+\Delta t} = X_t - \frac{\beta(t)}{2} X_t \Delta t + \sqrt{\beta(t) \Delta t} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I). \quad (2.12)$$

Anderson [8] demostró que la inversión temporal de toda difusión que satisface la ecuación de Fokker-Planck es en sí misma una difusión, con una deriva corregida por el *score* del marginal:

$$dX_t = \left[-\frac{\beta(t)}{2} X_t - \beta(t) \nabla_x \log p_t(X_t) \right] dt + \sqrt{\beta(t)} d\bar{W}_t, \quad (2.13)$$

La demostración consiste en escribir la ecuación de Fokker-Planck asociada al proceso inverso y verificar que $p(x, T-t)$ la satisface siempre que p_t satisfaga la ecuación de Fokker-Planck. El único término desconocido en la SDE backward es la función *score* $\nabla_x \log p_t(x)$, aproximada por una red neuronal $s_\theta(x, t)$ entrenada por *denoising score matching* [6]. Como el cronograma de transición es gaussiano, $p_t(x_t | x_0) = \mathcal{N}(x_t; \alpha_t x_0, \sigma_t^2 I)$, el *score* condicional se obtiene tomando el gradiente de la log-densidad:

$$\nabla_{x_t} \log p_t(x_t | x_0) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\sigma_t \epsilon}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}, \quad (2.14)$$

donde en el último paso se sustituye $x_t - \alpha_t x_0 = \sigma_t \epsilon$ a partir de la definición del kernel. Por tanto, el objetivo de entrenamiento se reduce a la predicción del ruido:

$$\mathcal{L}(\theta) = \mathbb{E}_{t, X_0, \epsilon} \left[\|\epsilon_\theta(X_t, t) - \epsilon\|^2 \right]. \quad (2.15)$$

Partiendo de $X_T \sim \mathcal{N}(0, I)$ e iterando para $t = T, T - \Delta t, \dots, \Delta t$:

$$X_{t-\Delta t} = X_t + \left[\frac{\beta(t)}{2} X_t + \beta(t) s_\theta(X_t, t) \right] \Delta t + \sqrt{\beta(t) \Delta t} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I). \quad (2.16)$$

Tras $N = T/\Delta t$ pasos, X_0 es una muestra aproximadamente distribuida según p_0 .

2.3. Modelos de procesos de difusión para generación condicionada

2.3.1. El problema de la generación condicionada

La generación condicionada tiene como objetivo muestrear de la distribución condicionada $p(\mathbf{x}_0 | \mathbf{y})$ (en lugar de la distribución concreta $p(\mathbf{x}_0)$ donde el modelo entrenado no está sujeto a ninguna clase), donde $\mathbf{y}$ es una clase asociada a $\mathbf{x}_0$. Para ello, aplicando el Teorema de Bayes:

$$p(\mathbf{x}_0 | \mathbf{y}) = \frac{p(\mathbf{x}_0) p(\mathbf{y} | \mathbf{x}_0)}{p(\mathbf{y})}$$

En el marco de la difusión, extenderemos esta idea a cada nivel de ruido t :

$$p(\mathbf{x}_t | \mathbf{y}) = \frac{p(\mathbf{x}_t) p(\mathbf{y} | \mathbf{x}_t)}{p(\mathbf{y})}$$

Y por tanto el score que se pretende aprender ahora es:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{y}) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y} | \mathbf{x}_t)$$

que con el Teorema de Bayes queda de la forma:

$$\nabla_{\mathbf{x}_t} \log(p(\mathbf{x}_t) p(\mathbf{y} | \mathbf{x}_t)) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y} | \mathbf{x}_t) \quad (2.17)$$

2.3.2. Classifier guidance

Lo que proponen Dhariwal y Nichol (2021) en [17] es usar un truco adaptado de Song en [9] que aprovecha la conexión entre los modelos de difusión y el score matching. En particular, si tenemos un modelo $\epsilon_\theta(\mathbf{x}_t)$ (que se relaciona con el score estimado por la red neuronal mediante $s_\theta(\mathbf{x}_t, t) = -\frac{\epsilon_\theta(\mathbf{x}_t, t)}{\sigma_t}$) capaz de predecir el ruido añadido a una muestra, esto puede ser usado para derivar la función de score:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \approx -\frac{1}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t)$$

Y aplicando esto a (2.17) obtendríamos:

$$\nabla_{\mathbf{x}_t} \log(p(\mathbf{x}_t)p(\mathbf{y}|\mathbf{x}_t)) = -\frac{1}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

E introducen un ϵ_θ que correspondería al score de la distribución conjunta:

$$\epsilon_\theta(\mathbf{x}_t|\mathbf{y}) := \epsilon_\theta(\mathbf{x}_t) - \sqrt{1-\alpha_t} \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

Nota: como en la literatura de más adelante usan $\sqrt{1-\alpha_t} = \sigma_t$, lo adaptaremos de esta forma para que seguirlo sea más sencillo. Ho y Salinas (2022) [18], un año más tarde, para mejorar esto, proponen modificar $\epsilon_\theta(\mathbf{x}_t)$ tal que:

$$\epsilon_\theta(\mathbf{x}_t) \approx -\sigma_t \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y})$$

para incluir el gradiente de la log verosimilitud de un modelo de clasificación conjunta, denotado por $\log p(\mathbf{y}|\mathbf{x}_t)$, tal y como se sigue:

$$\epsilon_\theta(\mathbf{x}_t, t) := \sigma_t \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) - \sigma_t w \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) = \sigma_t [\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{y}) - w \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)] \quad (2.18)$$

Introduciendo el parámetro w que regula la fuerza del classifier guidance. Usando este score para muestrear resulta (idealmente) en muestras distribuidas de la forma:

$$p(\mathbf{x}_0|\mathbf{y}) \propto p(\mathbf{y}|\mathbf{x}_0)p(\mathbf{x}_0)$$

La literatura asume valores típicos de $w \in [1, 10]$. El uso de este parámetro supone un trade-off entre variedad de las imágenes generadas y correspondencia clara a la clase a la que pertenece. De esta forma, $w \approx 0$ implicaría mucha variedad pero generación de imágenes que no necesariamente pertenecen a la clase. Por el contrario, $w \approx 10$ reduce casi por completo la variedad asegurándose de introducir imágenes de la clase que se le espera.

2.3.3. Classifier-free guidance

En lugar de un clasificador por separado, Ho y Salimans [18] proponen entrenar un único modelo que reciba o bien $\mathbf{y}$ o bien $\emptyset$, que representa la denominada condición nula. De esta forma:

$$\epsilon_\theta(\mathbf{x}_t, t) := \epsilon_\theta(\mathbf{x}_t, t|\emptyset) + w[\epsilon(\mathbf{x}_t, t|\mathbf{y}) - \epsilon(\mathbf{x}_t, t|\emptyset)] \quad (2.19)$$

Esto resulta especialmente útil para poder calcular el gradiente del clasificador sin tener el clasificador. Al reemplazar la condición $\mathbf{y}$ para un subconjunto de las muestras, fuerzas al modelo a aprender también la distribución incondicional $p(\mathbf{x}_0)$. De esta forma, puede aprender $\epsilon_\theta(\mathbf{x}_t, t|\mathbf{y})$ así como $\epsilon_\theta(\mathbf{x}_t, t|\emptyset) \equiv \epsilon_\theta(\mathbf{x}_t, t)$

2.3.4. Función de pérdida

Como se presenta en [18], $\mathbf{x}_0$ se parametriza a partir de $\mathbf{x}_t$:

$$\mathbf{x}_0(\mathbf{x}_t) = \frac{\mathbf{x}_t - \sigma_t \epsilon_\theta(\mathbf{x}_t, t | \mathbf{y})}{\sqrt{\alpha_t}}$$

Y desarrollando brevemente la función de pérdida propuesta en la ecuación (2.1), tendríamos:

$$\ell(\theta; \alpha, \sigma, \epsilon) = \mathbb{E}_{\epsilon, t, \mathbf{x}_0} [\|\epsilon_\theta(\mathbf{x}_t, t | \mathbf{y}) - \epsilon\|_2^2] \quad (2.20)$$

con $\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sigma_t \epsilon$ y $\epsilon \sim \mathcal{N}(0, 1)$.

2.3.5. Planteamiento de las SDEs

SDE Forward

Para la SDE forward usaremos Variance Preserving (2.8) con un schedule de varianza que usa los parámetros $\alpha_t = e^{-\int_0^t \beta(s) ds}$ y $\sigma_t = \sqrt{1 - \alpha_t}$. Su versión discretizada por el integrador de Euler-Maruyama sería igual a la (2.10):

$$d\mathbf{x} = \left[-\frac{1}{2} \beta(t) \mathbf{x}_t + \beta(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) \right] dt + \sqrt{\beta(t)} dW \quad (2.21)$$

SDE backward

En este caso, recordemos que el score aprendido es de la forma:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \approx s_\theta(\mathbf{x}_t, t) = \frac{1}{\sigma_t} \epsilon_\theta(\mathbf{x}_t)$$

por lo que podemos escribir la SDE backward del VP de forma muy parecida a como la introducíamos en (2.9):

$$d\mathbf{x} = \left[-\frac{1}{2} \beta(t) \mathbf{x}_t + \frac{\beta(t)}{\sigma_t} \epsilon_\theta(\mathbf{x}_t, t) \right] dt + \sqrt{\beta(t)} dW$$

2.3.6. SDE backward condicionada

Tan solo habría que sustituir $\epsilon_\theta \rightarrow \epsilon_\theta(\cdot | \mathbf{y})$

$$d\mathbf{x} = \left[-\frac{1}{2} \beta(t) \mathbf{x}_t + \frac{\beta(t)}{\sigma_t} \epsilon_\theta(\mathbf{x}_t, t | \mathbf{y}) \right] dt + \sqrt{\beta(t)} dW$$

Y su versión discretizada por Euler-Maruyama sería:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \left[\frac{1}{2} \beta(t) \mathbf{x}_t + \frac{\beta(t)}{\sigma_t} \epsilon_\theta(\mathbf{x}_t, t | \mathbf{y}) \right] \Delta t + \sqrt{\beta(t) \Delta t} \mathbf{z}_t \quad (\text{Backward VP}) \quad (2.22)$$

2.3.7. Imputación

La imputación es el problema de generación condicionada subyacente a aquellas imágenes $\mathbf{x}_0 \in \mathbb{R}^{\mathcal{D}}$ de las que algunos píxeles son conocidos y otros no. La máscara $\mathbf{m} \in \{0, 1\}^{\mathcal{D}}$ indica cuales:

$$\mathbf{y} = \mathbf{m} \odot \mathbf{x}_0$$

Usar classifier-free guidance fue propuesto por Nichol (2022), [19]. Pero la formalización matemática fue en realidad introducida por [20], donde proponen que, de nuevo, el objetivo es muestrear de $p(\mathbf{x}_0|\mathbf{y})$, generando una imagen que sea consistente con los píxeles ya conocidos. Aplicando el Teorema de Bayes, obtendríamos la expresión (2.17); es particularmente útil volver a observarla:

$$\nabla_{\mathbf{x}_t} \log(p(\mathbf{x}_t)p(\mathbf{y}|\mathbf{x}_t)) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

pues ya podemos estimar el primer término, pero no el segundo. Para solo tener que hacer una estimación para $s_\theta(\mathbf{x}_t, t)$, en cada paso temporal t se hacen dos cosas:

1. El paso backward siguiendo la misma idea que en la ecuación (2.22).
2. Estrategia replacement:

$$\mathbf{x}_{t-1, \text{ real}} = \mathbf{m} \odot \mathbf{x}_{t-1, \text{ forward}} + (1 - \mathbf{m}) \odot \mathbf{x}_{t-1, \text{ backward}}$$

Nota: es claro que la notación usada es propia, no concuerda con la de ningún artículo. Si bien es menos profesional, la consideramos más autocontenida y descriptiva. Esta estrategia permite inyectar el ruido forward adecuadamente solo en los píxeles ya conocidos, y en los que están por corregir, inyectar la corrección del modelo. Esto, además de ser más eficiente que calcular $\nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$ por separado (de haber una forma), permite generar de una forma suave en los píxeles pertinentes, en lugar de forzar bordes incómodos poco naturales. En [21] proponen el uso de una función de pérdida concreta para este método:

$$\mathcal{L} = \mathbb{E}_{z_t, \epsilon, t} [\|\epsilon_\theta((\mathbf{m} \odot \mathbf{x}_s), \mathbf{z}_t, t, c) - \epsilon\|_2^2]$$

donde es fácil notar que $\mathbf{x}_s$ denota la imagen, $\mathbf{z}_t$ su ruido a tiempo t y c un token para evitar efecto 'copia-pegar' entre distintas imágenes. Sin embargo, como se desarrolla en el proyecto, la función de pérdida convencional (la misma que para generación condicionada) sintetizada en (2.20) funciona prácticamente igual.

2.4. Fenómeno del double descent

El fenómeno del double descent constituye una revisión fundamental de la comprensión clásica de la generalización en aprendizaje automático. La teoría estadística tradicional describe la relación entre complejidad del modelo y error de generalización mediante una curva en forma de U: a medida que aumenta la capacidad del modelo, el sesgo disminuye pero la varianza crece, existiendo un óptimo más allá del cual el sobreajuste degrada el rendimiento. Belkin [22] demuestra que esta imagen es incompleta: si se sigue aumentando la capacidad más allá del punto en que el modelo interpola exactamente los datos de entrenamiento, el error de generalización vuelve a descender, a menudo por debajo del mínimo clásico.

El punto crítico es el umbral de interpolación, definido como el valor de complejidad a partir del cual el modelo alcanza error de entrenamiento cero. En su entorno, el modelo se ve forzado a memorizar los datos perfectamente, produciendo el pico característico del error de test. A la derecha del umbral, la sobreparametrización permite encontrar soluciones de norma mínima entre todas las que interpolan los datos, y son estas soluciones las que exhiben buenas propiedades de generalización.

Nakkiran [23] generaliza el fenómeno introduciendo la Effective Model Complexity (EMC): el número máximo de muestras sobre las que un procedimiento de entrenamiento concreto (arquitectura, optimizador y número de pasos) puede alcanzar error próximo a cero. Bajo esta definición, el *double descent* se manifiesta en tres ejes:

1. Model-wise. Con muestras y épocas fijos, el error de test en función de la capacidad arquitectónica sigue la curva de doble descenso, con el pico situado sistemáticamente en el umbral de interpolación. Este comportamiento se observa en CIFAR-10, CIFAR-100 e IWSLT'14, con CNNs, ResNets y Transformers en [23].

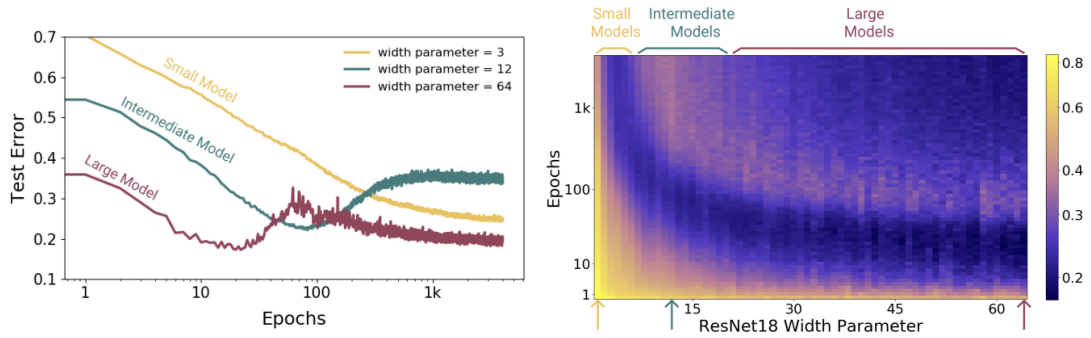


Figura 2.2: Representación del fenómeno de doble descenso *epoch-wise* según el estudio de [23]. Se muestra cómo el error de prueba puede exhibir un comportamiento no monótono a lo largo de las épocas de entrenamiento en regímenes cercanos al umbral de interpolación.

2. Epoch-wise. Con arquitectura y datos fijos, el error de test primero decrece, luego aumenta al alcanzar error de entrenamiento cero, y finalmente vuelve a decrecer. La implicación práctica es que el early stopping en el primer mínimo no es necesariamente óptimo.

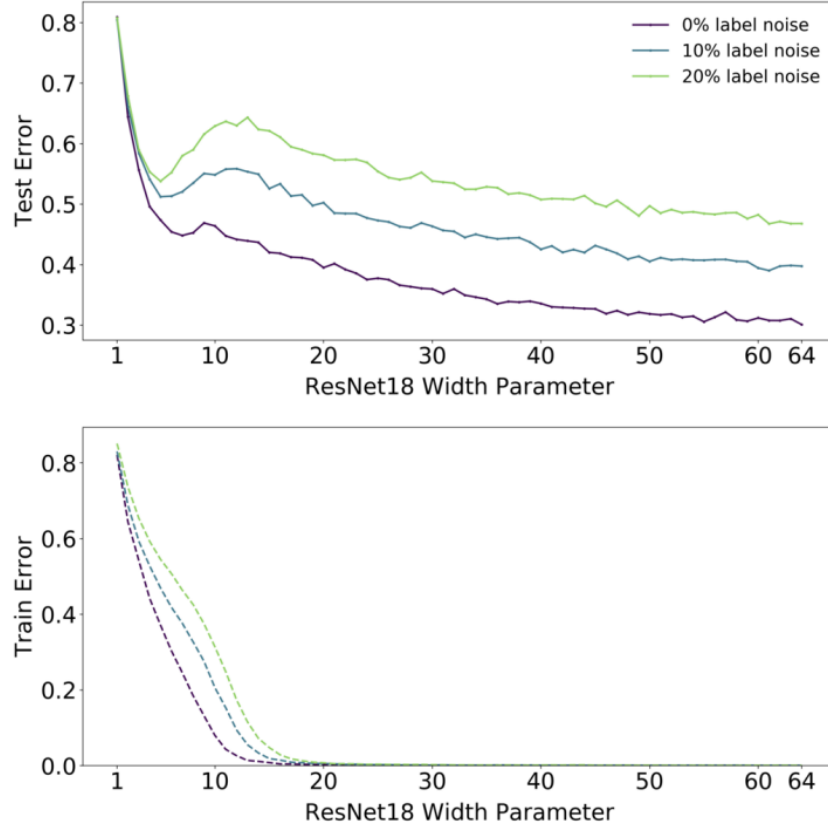


Figura 2.3: Evolución del error de prueba en función de la capacidad del modelo (*model-wise double descent*) adaptado de [23]. Se observa el pico de error en el umbral de interpolación y el posterior descenso en la zona de sobre-parametrización.

3. Sample-wise non-monotonicity. En la región crítica próxima al umbral de interpolación, aumentar el número de muestras puede desplazar el modelo hacia esa zona de alta sensibilidad, degradando el rendimiento en test. Los autores distinguen este efecto del *double descent* propiamente dicho, pues no presenta necesariamente la estructura descenso-ascenso-descenso de los otros dos ejes.

En conjunto, la complejidad efectiva de un modelo no puede reducirse al número de parámetros, sino que depende del procedimiento de entrenamiento completo. A lo largo de este trabajo se emplea el término *double descent* para referirse a cualquiera de estas manifestaciones, siguiendo la hipótesis generalizada de [23].

Capítulo 3

Desarrollo de Software y Diseño

3.1. Análisis de Requisitos

Los siguientes requisitos se han definido de manera específica para permitir la ejecución de tests automatizados que demuestren su cumplimiento.

3.1.1. Requisitos funcionales

- **RF-01: Configuración de la librería y parámetros globales.** El sistema debe actuar como un paquete de software integrado en Python que exponga una interfaz de programación para inicializar y configurar los hiperparámetros fundamentales de la generación, permitiendo definir el tipo de proceso estocástico a instanciar antes de iniciar el entrenamiento o la inferencia.
- **RF-02: Instanciación adaptativa de arquitecturas U-Net.** El sistema debe instanciar dinámicamente modelos de redes neuronales basados en la arquitectura U-Net, ajustando la dimensionalidad de las capas de entrada y salida para procesar tensores correspondientes a imágenes en escala de grises de 1 canal o imágenes a color en formato RGB de 3 canales.
- **RF-03: Implementación de cronogramas de ruido (*Schedules*).** El sistema debe calcular y exponer cronogramas de variación de ruido a lo largo de la trayectoria de difusión temporal, permitiendo al usuario inyectar la perturbación siguiendo un comportamiento escalar estrictamente lineal, un decaimiento basado en la función coseno o una curva exponencial.
- **RF-04: Ejecución de difusión *forward* mediante Variance Exploding (VE).** El sistema debe perturbar las imágenes empíricas inyectando ruido gaussiano continuo según la formulación estocástica VE, manipulando el tensor de entrada para que su varianza diverja hacia el infinito conforme el tiempo de difusión avanza, garantizando un límite de perturbación máxima configurado.
- **RF-05: Ejecución de difusión *forward* mediante Variance Preserving (VP) según el *schedule* escogido.** El sistema debe proveer de forma alternativa la formulación de difusión VP, escalando los datos originales de entrenamiento y sumando ruido de forma proporcional iterativa para garantizar que la varianza total de la distribución se mantenga matemáticamente acotada alrededor de la unidad en todo momento.

- **RF-06: Procesamiento de entrenamiento continuo por lotes.** El sistema debe poder implementar el bucle de optimización del modelo U-Net (según el DataLoader, batch size, learning rate, optimizador u otros parámetros del sistema que elija el usuario) inyectando ruido en un lote de imágenes basado en instantes temporales aleatorizados, calculando la función de pérdida mediante *score matching* al comparar el gradiente real del ruido inyectado con el *score* bidimensional estimado por la red neuronal.
- **RF-07: Muestreo e integración *backward* (SDE y ODE).** Para la síntesis de nuevas muestras a partir del espacio latente, el sistema debe implementar y exponer un módulo de muestreadores numéricos que resuelvan la trayectoria de difusión inversa, permitiendo al usuario seleccionar dinámicamente entre aproximaciones estocásticas que resuelven la Ecuación Diferencial Estocástica (SDE) (incluyendo el esquema de Euler-Maruyama y el método iterativo Predictor-Corrector regulado por un parámetro configurable de relación señal-ruido (SNR) objetivo), o un enfoque estrictamente determinista que integre la Ecuación Diferencial Ordinaria de flujo de probabilidad (*Probability Flow ODE*) para garantizar un mapeo exacto y continuo entre el ruido puro y la distribución empírica de los datos.
- **RF-08: Condicionamiento semántico (*Classifier-Free Guidance*).** El sistema debe permitir el control direccional sobre el proceso de muestreo mediante CFG, modificando el vector de *score* predictivo efectivo a través de una interpolación o extrapolación lineal entre una predicción condicionada a una clase de entrada y una predicción incondicional, ponderada por una magnitud escalar ajustable.
- **RF-09: Imputación estructural basada en máscaras (*Inpainting*).** El sistema debe resolver problemas de visión espacial reconstruyendo imágenes parcialmente ocluidas; para ello, debe recibir un tensor de imagen deteriorado junto con una máscara binaria y forzar la inyección de los valores conocidos de los píxeles intactos en cada paso de la discretización *backward*.
- **RF-10: Evaluación estadística de densidad (BPD).** El sistema debe disponer de un módulo evaluador capaz de procesar las distribuciones de los tensores de imágenes generadas para estimar y retornar los Bits por Dimensión (BPD), aportando una métrica directa de la verosimilitud de las muestras creadas frente al colector de datos original.
- **RF-11: Extracción y evaluación de fidelidad visual (FID e IS).** El sistema debe integrar internamente un modelo preentrenado de extracción de características profundas, específicamente la red InceptionV3, para mapear las distribuciones de lotes masivos de imágenes de síntesis y de referencia empírica con el fin de calcular y emitir el *Fréchet Inception Distance* (FID) y el *Inception Score* (IS).

3.1.2. Requisitos no funcionales

- **RNF-01: Entorno y herramientas.** El sistema debe desarrollarse en Python, integrarse con frameworks de tensores como PyTorch o TensorFlow, y documentar sus dependencias clave para asegurar la reproducibilidad del entorno.
- **RNF-02: Testabilidad automatizada.** El diseño debe garantizar el aislamiento de los componentes para permitir la ejecución sistemática de tests automatizados que demuestren empíricamente el cumplimiento de los requisitos funcionales.

- **RNF-03: Arquitectura modular.** El paquete `diffusion_lib` debe estructurarse mediante Clases Base Abstractas (ABC) para asegurar un bajo acoplamiento y permitir la extensión de muestreadores, procesos y cronogramas.
- **RNF-04: Rendimiento y aceleración.** Los algoritmos estocásticos y el entrenamiento de la U-Net deben estar optimizados para vectorizar operaciones y aprovechar la aceleración por hardware (GPU).
- **RNF-05: Usabilidad de la API.** El sistema debe exponer una interfaz de programación estandarizada, robusta y con anotaciones de tipado para facilitar la configuración de parámetros a los usuarios finales.

3.2. Casos de Uso

- **CU-01: Entrenamiento de modelos predictivos de *score*.** Un investigador o desarrollador configura un flujo de entrenamiento aportando un conjunto de datos masivo con imágenes de cualquier dominio visual imaginable. El sistema inyecta ruido progresivo mediante las formulaciones estocásticas VE o VP y optimiza la red U-Net subyacente para que aprenda a revertir la perturbación sobre dicha distribución específica.
- **CU-02: Generación incondicional de imágenes.** A partir de un modelo previamente entrenado y sin necesidad de *prompts* adicionales, el usuario solicita la creación de un lote de muestras nuevas. El sistema inicializa tensores de ruido gaussiano puro y aplica sus integradores numéricos *backward* (SDE u ODE) para decodificar ese ruido en imágenes únicas, realistas y coherentes con el conjunto de datos de origen.
- **CU-03: Síntesis visual condicionada por categoría.** El usuario requiere generar elementos visuales concretos dentro del dominio aprendido por el modelo. Introduciendo un identificador de clase y ajustando la escala de *Classifier-Free Guidance* (CFG), el sistema dirige el muestreo *backward* para sintetizar exclusivamente imágenes que cumplan con la semántica solicitada, rechazando características de otras clases.
- **CU-04: Restauración e imputación de regiones (*Inpainting*).** El usuario introduce una imagen deteriorada, censurada o con áreas de información faltante, acompañada de una máscara binaria indicadora. El sistema aprovecha el conocimiento estructural de la U-Net para ejecutar un proceso de difusión condicionada espacialmente, rellenando los vacíos visuales de forma fluida y coherente con los píxeles adyacentes conocidos.
- **CU-05: Auditoría y evaluación comparativa de la calidad generativa.** Tras completar diversas fases de entrenamiento o al probar nuevos muestreadores, el usuario solicita un análisis de rendimiento. El sistema contrasta automáticamente un lote de imágenes recién sintetizadas contra la distribución del conjunto de datos real, emitiendo un informe cuantitativo que incluye el BPD, FID e IS para evaluar objetivamente la fidelidad visual y diversidad del modelo.

3.3. Arquitectura y Diseño

3.3.1. Estructura del paquete

El sistema se organiza como un paquete Python instalable con la siguiente estructura de módulos:

```
diffusion_lib/
├── model.py .....Meta-clase para un proceso de difusión
├── processes/ .....Procesos de difusión
│   ├── base.py .....BaseProcess (ABC)
│   ├── ve.py .....Variance Exploding
│   └── vp.py .....Variance Preserving
├── samplers/ .....Integradores de la SDE inversa
│   ├── base.py .....BaseSampler (ABC)
│   ├── euler_maruyama.py
│   ├── predictor_corrector.py
│   ├── probability_flow_ode.py
│   └── imputation.py
├── schedules/ .....Noise schedules
│   ├── base.py .....BaseSchedule (ABC)
│   ├── linear.py
│   └── cosine.py
├── metrics/ .....Métricas de calidad
│   ├── bpd.py .....Bits per dimension
│   └── fid_is.py .....FID e IS sobre InceptionV3
└── score_models/ .....Distintas U-Nets
    ├── base.py
    ├── cfg_wrapper.py .....Envuelve el score model condicional
    ├── conditional.py
    └── unconditional.py
```

Figura 3.1: Estructura del paquete `diffusion_lib`

3.4. Validación y Calidad del Software

3.4.1. Metodología de desarrollo

El desarrollo de `diffusion_lib` ha seguido un flujo de trabajo basado en feature branches sobre Git. Cada nueva funcionalidad (proceso de difusión, muestreador, cronograma o módulo de métricas) se desarrolla en una rama independiente que se integra en `main` únicamente tras superar la suite de tests automáticos y la revisión de código.

3.4.2. Guías de estilo

El código fuente sigue el estándar PEP 8 [24]. Las reglas de formateo se delegan en Black con una longitud de línea máxima de 88 caracteres, y el orden de las importaciones se gestiona con `isort` configurado en modo compatible con Black. Se emplean anotaciones de tipo `type hints` en las firmas de las clases, siguiendo las recomendaciones de PEP 484 [24]. Los docstrings adoptan el estilo Google y se aplican a nivel de clase para documentar parámetros, comportamiento matemático y referencias bibliográficas.

3.4.3. Tests automatizados

La suite de pruebas se encuentra en el directorio `tests/`, con una estructura que espeja la del paquete:

```
tests/  
    test_processes.py  
    test_schedules.py  
    test_score_models.py  
    test_model.py
```

Los tests se organizan en tres niveles:

1. Unit tests: verifican la corrección matemática de funciones individuales (por ejemplo, que la varianza del proceso VE crece monótonamente con t , o que el cronograma coseno cumple $\bar{\alpha}_0 \approx 1$ y $\bar{\alpha}_T \approx 0$).
2. Integration tests: comprueban la coherencia entre módulos ejecutando ciclos cortos de entrenamiento y muestreo sobre datos sintéticos de baja resolución (8×8 , CPU).
3. Smoke tests: instancian cada clase pública para detectar errores de importación o firma. Para los componentes que involucran PyTorch, se utiliza un `torch.manual_seed` fijo en cada test que requiere reproducibilidad. El objetivo de cobertura mínima establecido en `.coveragerc` es del 80 %.

Capítulo 4

Desarrollo del proyecto y resultados

4.1. VE vs VP

1. Justificar por qué elegimos el schedule `jschedule` y por qué vamos a usar EM en general (podríamos usar otros, pero por simplicidad usaremos EM, luego comparamos EM con PFODE). 2. Insertar una comparativa visual de la generación de imágenes y desarrollar rápidamente cuál creemos que se ve mejor y por qué. 3. Medición del FID.

4.2. Schedule lineal vs Schedule coseno en VP

4.3. VE / VP vs Fokker Planck

4.4. Generación condicionada

4.5. Medición de métricas en test para los checkpoints de los gatos.

Capítulo 5

Conclusiones

TODO: Conclusiones sobre el trabajo realizado

Bibliografía

- [1] K. P. Murphy, *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.
- [2] R. Vershynin, *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2nd ed., 2026.
- [3] L. Weng, “What are diffusion models?,” *lilianweng.github.io*, Jul 2021.
- [4] M. Welling and Y. W. Teh, “Bayesian learning via stochastic gradient langevin dynamics,” in *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 681–688, ACM, 2011.
- [5] A. Hyvärinen, “Estimation of non-normalized statistical models by score matching,” *Journal of Machine Learning Research*, vol. 6, no. 24, pp. 695–709, 2005.
- [6] P. Vincent, “A connection between score matching and denoising autoencoders,” Tech. Rep. 1358, Department of Computer Science and Operations Research, University of Montreal, 2010.
- [7] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [8] B. D. O. Anderson, “Reverse-time diffusion equation models,” *Stochastic Processes and their Applications*, vol. 12, no. 3, pp. 313–326, 1982.
- [9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations*, 2021.
- [10] Z. Nasir, “Cosine-beta schedule for denoising diffusion models.” <https://www.zainnasir.com/blog/cosine-beta-schedule-for-denoising-diffusion-models/>, 2023. Accessed: 2026-04-22.
- [11] T. Hang, S. Gu, X. Geng, and B. Guo, “Improved noise schedule for diffusion training,” Tech. Rep. 2407.03297, arXiv, 2024.
- [12] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *Proceedings of the 38th International Conference on Machine Learning* (M. Meila and T. Zhang, eds.), vol. 139 of *Proceedings of Machine Learning Research*, pp. 8162–8171, PMLR, 2021.
- [13] H. Risken, *The Fokker–Planck Equation: Methods of Solution and Applications*. Springer, 2nd ed., 1989.

- [14] C. W. Gardiner, *Handbook of Stochastic Methods for Physics, Chemistry and the Natural Sciences*, vol. 13 of *Springer Series in Synergetics*. Springer-Verlag, 1994.
- [15] B. Øksendal, *Stochastic Differential Equations: An Introduction with Applications*. Springer, 6th ed., 2003.
- [16] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.
- [17] P. Dhariwal and A. Nichol, “Diffusion models beat GANs on image synthesis,” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 8780–8794, 2021.
- [18] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [19] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever, and M. Chen, “Glide: Towards photorealistic image generation and editing with text-guided diffusion models,” in *International Conference on Machine Learning*, pp. 16784–16804, PMLR, 2022.
- [20] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool, “Repaint: Inpainting using denoising diffusion probabilistic models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11461–11471, 2022.
- [21] M. Ozturk *et al.*, “Enriching image conditioned inpainting in latent diffusion models for virtual try-all,” 2024.
- [22] M. Belkin, D. Hsu, S. Ma, and S. Mandal, “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, pp. 15849–15854, 2019.
- [23] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever, “Deep double descent: Where bigger models and more data hurt,” 2019.
- [24] G. van Rossum, B. Warsaw, and N. Coghlan, “PEP 8 – Style Guide for Python Code,” 2001.
- [25] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [26] H. Narayanan and S. Mitter, “Sample complexity of testing the manifold hypothesis,” in *Advances in Neural Information Processing Systems*, vol. 23, 2010.
- [27] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pp. 234–241, Springer, 2015.
- [28] S. Zagoruyko and N. Komodakis, “Wide residual networks,” *arXiv preprint arXiv:1605.07146*, 2016.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017.
- [30] A. Suárez, “Stochastic differential equations: Diffusion models and generative ai.” Apuntes de clase, Curso de Aprendizaje Automático, 2026. Documento disponible en el Moodle institucional.

- [31] L. Theis, A. van den Oord, and M. Bethge, “A note on the evaluation of generative models,” 2015.
- [32] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017.
- [33] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.
- [34] M. Fréchet, “Sur la distance de deux lois de probabilité,” *Comptes Rendus de l’Académie des Sciences*, vol. 244, pp. 689–692, 1957.
- [35] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4195–4205, 2023.
- [36] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton, “Demystifying MMD GANs,” *arXiv preprint arXiv:1801.01401*, 2018.
- [37] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved techniques for training GANs,” in *Advances in Neural Information Processing Systems*, vol. 29, Curran Associates, Inc., 2016.
- [38] S. Barratt and R. Sharma, “A note on the inception score,” *arXiv preprint arXiv:1801.01973*, 2018.
- [39] T. Kynkäänniemi, T. Karras, M. Aittala, T. Aila, and J. Lehtinen, “The role of ImageNet classes in Fréchet Inception Distance,” in *arXiv preprint arXiv:2203.06026*, 2022.
- [40] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila, “Improved precision and recall metric for assessing generative models,” in *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019.

Autoencoders, espacios latentes y manifold hypothesis

Autoencoders y espacios latentes Los autoencoders son redes neuronales profundas cuya motivación surge de tratar conseguir réplicas en forma de output de ciertos inputs. En su arquitectura presentan una capa interna denotada por h que describe un código concreto acerca de $\mathbf{x}$ en una dimensión a priori menor que la de $\mathbf{x}$ (undercomplete autoencoders). Estas arquitecturas están pensadas entonces para tener 2 partes: la de codificación $\mathbf{h} = f(\mathbf{x})$, $f : \mathbb{R}^{\mathcal{D}} \rightarrow \mathbb{R}^{\mathcal{L}}$ con $\mathcal{D} > \mathcal{L}$ y la de reconstrucción $\mathbf{r} = g(\mathbf{h})$, $g : \mathbb{R}^{\mathcal{L}} \rightarrow \mathbb{R}^{\mathcal{D}}$. Se denota por espacio latente al espacio generado en el paso intermedio, el espacio en que vive h , $h \in \mathbb{R}^{\mathcal{L}}$ [25].

Manifold hypothesis Se entiende a la manifold hypothesis por la teoría que defiende que las distribuciones de ciertos tipos de datos concretos como imágenes, textos o sonidos naturales se encuentra altamente concentrada [25]. Es por ello que se recurre a los espacios latentes para codificar estas distribuciones de probabilidad, puesto que es en estos espacios de dimensionalidad menor donde puede codificarse la información relevante acerca de los datos. Existen pruebas sofisticadas a favor de esta hipótesis, concentrados, entre otros, en [26].

Apéndice: Gaussiana Mutivariada, Divergencia Kullback Leibler y ELBO

Gaussiana Multivariada

Sea $\mathbf{x} \in \mathbb{R}^D$. Decimos que $\mathbf{x}$ sigue una distribución gaussiana multivariada con parámetros $\mu \in \mathbb{R}^D$, $\Sigma \in \mathbb{R}^{D \times D}$ y escribimos $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ si su función de densidad de probabilidad es:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left[-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right]$$

donde Σ es definida positiva, $|\Sigma|$ denota su determinante, μ es su media i.e. $\mathbb{E}[\mathbf{x}] = \mu$. Diremos que es isotrópica cuando $\Sigma = \sigma^2 \mathbb{I}$ para algún $\sigma^2 > 0$.

Divergencia Kullback-Leibler

Sean p y q dos distribuciones de probabilidad continuas sobre $\mathbb{R}^D$. La divergencia KL de q respecto a p se define como:

$$D_{\text{KL}}(q\|p) = \int_{\mathbb{R}^D} q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})} d\mathbf{x}$$

(*Propiedades de la KL*). La divergencia KL satisface:

1. $D_{\text{KL}}(q\|p) \geq 0$, $\forall p, \forall q$
2. $D_{\text{KL}}(q\|p) = 0 \iff p = q$
3. $D_{\text{KL}}(q\|p) \neq D_{\text{KL}}(p\|q)$ en general, i.e. no es simétrica

Apunte: maximizar la divergencia de Kullback-Leibler es equivalente a maximizar la log-verosimilitud o minimizar la log-verosimilitud negativa.

ELBO

Sea $q(\mathbf{z}|\mathbf{x})$ una distribución auxiliar aproximada. Entonces:

$$\log p_{\theta}(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))}_{\mathcal{L}(\theta; \mathbf{x}) := \text{ELBO}}$$

Demostración. Por la desigualdad de Jensen ([1]) aplicada a log, que es cóncava:

$$\log p_{\theta}(\mathbf{x}) = \log \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z} = \log \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right] \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right]$$

que expandiendo el logaritmo da la expresión del ELBO. $\square$

La ELBO es el objetivo de entrenamiento central en los modelos de difusión: maximizarla equivale a minimizar $D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})\|p_{\theta}(\mathbf{z}|\mathbf{x}))$, es decir, a hacer que la distribución aproximada sea lo más cercana posible a la posterior real.

Apéndice: U-Net

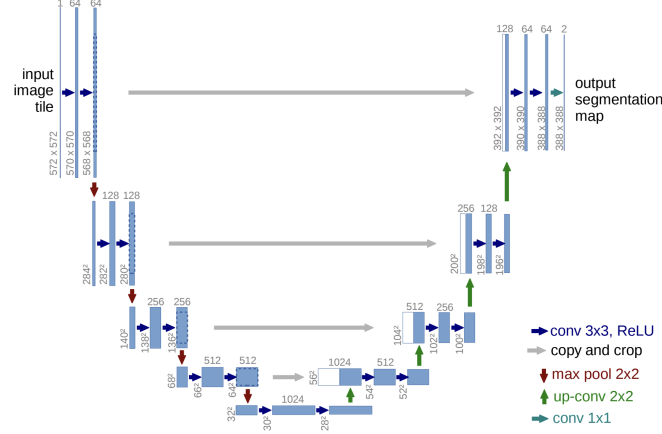


Figura 1: Arquitectura de la red U-Net [27]. La rama codificadora (izquierda) reduce progresivamente la resolución espacial mediante bloques de convolución y max-pooling, incrementando la profundidad de canales en cada nivel. La rama decodificadora (derecha) recupera la resolución original mediante convoluciones transpuestas. Las *skip-connections* (flechas grises) concatenan los mapas de características de cada nivel del codificador con su correspondiente en el decodificador.

La U-Net descrita fue concebida para segmentación semántica, sin necesidad de distinguir niveles de ruido. Para su uso en modelos de difusión, Ho et al. [16] introducen tres modificaciones principales:

- (i) los bloques de convolución se reemplazan por bloques residuales Wide ResNet [28] con *group normalization*;
- (ii) el paso temporal t se codifica mediante embeddings sinusoidales adaptados del Transformer [29], que se suman a la activación interna de cada bloque residual, permitiendo que la red distinga el nivel de ruido en cada instante;
- (iii) se añaden bloques de *multi-head self-attention* a resolución 16×16 , entre los bloques residuales, para capturar dependencias de largo alcance. Las skip-connections entre codificador y decodificador se mantienen sin cambios.

Apéndice: De denoising score matching individual al objetivo combinado multi-escalaz

En esta sección detallamos la deducción matemática de la función de pérdida empírica, partiendo de la formulación del proceso de difusión hacia adelante (*forward process*) hasta llegar a la expresión final dependiente de la variable de ruido $\mathbf{Z}$.

1. Distribución condicional y función *Score*

Dada una condición inicial de los datos $\mathbf{x}(0) = \mathbf{x}_0 \in \mathbb{R}^D$, la solución marginal del proceso hacia adelante en el tiempo t viene dada por:

$$\mathbf{x}(t) \mid \mathbf{x}_0 = \boldsymbol{\mu}_{\mathbf{x}_0}(t) + \sigma(t)\mathbf{Z} \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{x}_0}(t), \sigma^2(t)\mathbf{I}) \quad (1)$$

donde $\mathbf{Z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ es ruido gaussiano estándar. La función de densidad de probabilidad condicional $p_{t|0}(\mathbf{x}(t), t \mid \mathbf{x}_0, 0)$ toma la forma explícita de una gaussiana multivariante:

$$p_{t|0}(\mathbf{x}(t), t \mid \mathbf{x}_0, 0) = \frac{1}{(2\pi)^{D/2}} \exp\left(-\frac{1}{2\sigma^2(t)} (\mathbf{x}(t) - \boldsymbol{\mu}_{\mathbf{x}_0}(t))^\top (\mathbf{x}(t) - \boldsymbol{\mu}_{\mathbf{x}_0}(t))\right) \quad (2)$$

El primer paso para construir la función de pérdida es calcular la función *score* analítica de esta distribución de transición, es decir, el gradiente de su logaritmo respecto a $\mathbf{x}(t)$. Tomando el logaritmo de la densidad obtenemos:

$$\log p_{t|0}(\mathbf{x}(t), t \mid \mathbf{x}_0, 0) = C - \frac{1}{2\sigma^2(t)} \|\mathbf{x}(t) - \boldsymbol{\mu}_{\mathbf{x}_0}(t)\|_2^2$$

Calculando el gradiente con respecto a $\mathbf{x}(t)$, se tiene:

$$\nabla_{\mathbf{x}} \log p_{t|0}(\mathbf{x}(t), t \mid \mathbf{x}_0, 0) = -\frac{1}{\sigma^2(t)} (\mathbf{x}(t) - \boldsymbol{\mu}_{\mathbf{x}_0}(t)) \quad (3)$$

A partir de la definición inicial del proceso continuo, sabemos que la diferencia entre la variable ruidosa y su media es exactamente el término de varianza por el ruido inyectado, es decir, $\mathbf{x}(t) - \boldsymbol{\mu}_{\mathbf{x}_0}(t) = \sigma(t)\mathbf{Z}$. Sustituyendo esto en la ecuación anterior, obtenemos la deducción fundamental del *score*:

$$\nabla_{\mathbf{x}} \log p_{t|0}(\mathbf{x}(t), t \mid \mathbf{x}_0, 0) = -\frac{1}{\sigma^2(t)} (\sigma(t)\mathbf{Z}) = -\frac{1}{\sigma(t)} \mathbf{Z} \quad (4)$$

2. Desarrollo de la Función de Pérdida

El objetivo fundamental del aprendizaje de representaciones (*score matching*) es entrenar una red neuronal $\mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta})$ para aproximar el verdadero *score* de los datos. La formulación general de la función de pérdida ponderada en función del tiempo es:

$$loss(\boldsymbol{\theta}) = \mathbb{E}_{t \sim U[0,1]} \left[\lambda(t) \mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p_{t|0}(\mathbf{x}, t \mid \mathbf{x}_0, 0)} \left[\|\mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) - \nabla_{\mathbf{x}} \log p_{t|0}(\mathbf{x}, t \mid \mathbf{x}_0, 0)\|_2^2 \right] \right] \right]$$

Paso 1: Sustitución de la función *score*.

Sustituyendo el gradiente exacto del logaritmo derivado en la Ecuación 4 dentro del término de la norma:

$$\begin{aligned}
loss(\boldsymbol{\theta}) &= \mathbb{E}_{t \sim U[0,1]} \left[\lambda(t) \mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p_{t|0}} \left[\left\| \mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) - \left(-\frac{1}{\sigma(t)} \mathbf{Z} \right) \right\|_2^2 \right] \right] \right] \\
&= \mathbb{E}_{t \sim U[0,1]} \left[\lambda(t) \mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p_{t|0}} \left[\left\| \mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) + \frac{1}{\sigma(t)} \mathbf{Z} \right\|_2^2 \right] \right] \right] \quad (5)
\end{aligned}$$

Esta es la segunda expresión que se observa en el planteamiento de la pérdida.

Paso 2: Reagrupación escalar del término de ruido.

Para llegar a la formulación final, debemos homogeneizar la expresión dentro de la norma para poder extraer términos constantes. Multiplicamos y dividimos todo el contenido de la norma en la Ecuación 5 por $\sigma(t)$:

$$\left\| \mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) + \frac{1}{\sigma(t)} \mathbf{Z} \right\|_2^2 = \left\| \frac{1}{\sigma(t)} \left(\sigma(t) \mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) + \mathbf{Z} \right) \right\|_2^2$$

Aplicando la propiedad de la norma $\|c\mathbf{v}\|_2^2 = c^2 \|\mathbf{v}\|_2^2$ (donde c es un escalar, en este caso $1/\sigma(t)$), podemos extraer el coeficiente y nos queda una ponderación de $1/\sigma^2(t)$. Introduciendo además el peso temporal $\lambda(t)$ dentro de las esperanzas más internas debido a la linealidad de la esperanza, llegamos a la tercera y última expresión:

$$loss(\boldsymbol{\theta}) = \mathbb{E}_{t \sim U[0,1]} \left[\mathbb{E}_{\mathbf{x}_0 \sim p_0(\mathbf{x})} \left[\mathbb{E}_{\mathbf{x} \sim p_{t|0}(\mathbf{x}, t | \mathbf{x}_0, 0)} \left[\frac{\lambda(t)}{\sigma^2(t)} \left\| \sigma(t) \mathbf{s}(\mathbf{x}, t; \boldsymbol{\theta}) + \mathbf{Z} \right\|_2^2 \right] \right] \right]$$

Nota final: Tal y como se refleja en las ecuaciones de optimización, una elección estándar para el hiperparámetro de peso $\lambda(t)$ se basa en hacerlo inversamente proporcional a la esperanza del *score* al cuadrado. Es decir, asumiendo $\lambda(t) \propto \sigma^2(t)$, el prefactor $\frac{\lambda(t)}{\sigma^2(t)}$ se cancela, simplificando la función a una regresión que trata de escalar la salida de la red para predecir directamente el vector de ruido $\mathbf{Z}$. Desarrollo completamente desarrollado a partir de [30].

Apéndice: Movimiento Browniano y proceso de Wiener para SDE (Stochastic Differential Ecuations)

La forma más general de una ecuación diferencial estocástica fue propuesta por Itô es:

$$d\mathbf{x} = a(\mathbf{x}(t), t)dt + b(\mathbf{x}(t), t)dW$$

donde dW se define como un proceso de Wiener. Gardiner, en [14] plantea de forma genérica la ecuación de Langevin (que se verá más adelante en este proyecto) como:

$$\frac{d\mathbf{x}}{dt} = a(\mathbf{x}, t) + b(\mathbf{x}, t)\xi(t)$$

donde $\xi(t)$ es una variable aleatoria que fluctúa con rapidez. Gardiner argumenta por qué la definición de $\xi(t)$ genera problemas (no se entrará a ese nivel de detalle) y plantea una definición formal para esta variable de forma que sea integrable:

$$u(t) = \int_0^t dt' \xi(t')$$

donde $u(t)$ no depende de valores anteriores y es un proceso de Markov. Usando esta nueva definición, plantea la ecuación diferencial de Langevin como:

$$x(t) - x(0) = \int_0^t a[x(s), s]ds + \int_0^t b[x(s), s]\xi(s)ds$$

Y esto implica que podemos interpretar la integral de este proceso, es decir, la suma acumulada de ruido aleatorio (ruido blanco) definida por ξ como un proceso de Wiener y definir:

$$dW(t) \equiv W(t + dt) - W(t) = \xi(t)dt$$

A raíz de esto, se define la familia de ecuaciones diferenciales estocásticas, descritas por ruido de Wiener, que generan movimiento Browniano de la forma:

$$d\beta(t) = \mu(\mathbf{x}(t), t)dt + \sigma(t)dW(t)$$

donde identificaremos los términos $\mu : \mathbb{R}^D \times [0, T] \rightarrow \mathbb{R}^D$ como de deriva/drift y a $\sigma : [0, T] \rightarrow \mathbb{R}^D$ como el de difusión [14].

Apéndice: Integración numérica de la SDE inversa: samplers

Una vez entrenada $s_\theta \approx \nabla_{\mathbf{x}} \log p_t$, generar una muestra consiste en integrar la SDE inversa (§2.2.3)

$$d\mathbf{x} = [f(\mathbf{x}, t) - g(t)^2 s_\theta(\mathbf{x}, t)] dt + g(t) d\bar{W}_t, \quad t : T \rightarrow \varepsilon, \quad (6)$$

con $\varepsilon > 0$ pequeño para evitar la singularidad del score cerca de los datos. Se implementan tres integradores [9].

Euler–Maruyama (EM). Discretización estocástica directa de (6) con $\Delta t < 0$:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + [f(\mathbf{x}_n, t_n) - g(t_n)^2 s_\theta(\mathbf{x}_n, t_n)] \Delta t + g(t_n) \sqrt{|\Delta t|} \mathbf{z}_n. \quad (7)$$

Orden fuerte $\mathcal{O}(\sqrt{|\Delta t|})$. Línea de base; el sesgo de discretización degrada la calidad con N pequeño.

Predictor–corrector (PC). Un paso de EM (predictor) seguido de M pasos de Langevin en t_{n+1} (corrector) que re proyectan la muestra sobre el marginal $p_{t_{n+1}}$:

$$\mathbf{x} \leftarrow \mathbf{x} + \epsilon s_\theta(\mathbf{x}, t_{n+1}) + \sqrt{2\epsilon} \mathbf{z}, \quad \epsilon = 2 \left(r \frac{\|\mathbf{z}\|}{\|s_\theta\|} \right)^2, \quad (8)$$

con SNR objetivo $r = 0,16$ [9]. Coste $M+1$ evaluaciones de red por paso; compensa el sesgo de EM y mejora FID a igual presupuesto.

Probability flow ODE (PF-ODE). Aplicando $\Delta p_t = \nabla \cdot (p_t \nabla \log p_t)$ a la Fokker–Planck (2.2.6), se reescribe como ecuación de continuidad de una ODE *determinista* con los mismos marginales:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) - \frac{1}{2} g(t)^2 s_\theta(\mathbf{x}, t), \quad (9)$$

que se integra con Euler (u órdenes superiores: Heun, RK, DPM-Solver). Al ser determinista admite cambio de variable instantáneo,

$$\log p_0(\mathbf{x}_0) = \log p_T(\mathbf{x}_T) + \int_0^T \nabla \cdot \tilde{f}_\theta dt, \quad \tilde{f}_\theta = f - \frac{1}{2} g^2 s_\theta, \quad (10)$$

base del cálculo exacto del BPD (§2.4) y de la inversión/interpolación de muestras.

Sampler	Determ.	Evals./paso	BPD	Calidad con N bajo
Euler–Maruyama	No	1	No	Baja
Predictor–corrector	No	$M+1$	No	Media–alta
Probability flow ODE	Sí	1	Sí	Media*

Tabla 1: Comparativa. Con integradores de orden superior la calidad del PF-ODE aumenta apreciablemente.

Apéndice: Métricas de calidad para las imágenes generadas

Negative log-likelihood (NLL)

Esta es la métrica por excelencia usada para cuantificar la calidad de un modelo de IA generativa. Imaginemos que tenemos un conjunto de imágenes de la forma

$$\mathbf{x} = \{0, \dots, 255\}^{\mathcal{D}}$$

cuya densidad de probabilidad es desconocida, $P(\mathbf{x})$, un conjunto de imágenes ruidosas $\mathbf{y} = \mathbf{x} + u$, con $u \sim [0, 1]^{\mathcal{D}}$ con densidad de probabilidad $p(\mathbf{y})$ y un modelo que predice la densidad de probabilidad $q(\mathbf{x})$. La negative log-likelihood se define por:

$$NLL = -\mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})}[\log p(\mathbf{x})]$$

La idea es evaluar imágenes reservadas para test:

$$\mathbf{x}_{\text{test}} = \{\mathbf{x}_i\}_{i=1}^N, \quad \mathbf{x}_i \in \{0, \dots, 255\}^{\mathcal{D}}$$

(que para este problema pueden ser las mismas que para train) sobre la densidad de probabilidad que estima el modelo, obteniendo una NLL menor cuanto mayor sea la estimación del modelo, es decir, cuanto mayor sea la log-verosimilitud de la imagen i -ésima estimada sobre la predicción del modelo:

$$NLL = -\frac{1}{N} \sum_{i=1}^N \log p(\mathbf{x}_i)$$

En un tal caso, podríamos intuir que

$$p(\mathbf{x}) \approx P(\mathbf{x})$$

y que por tanto el modelo es capaz de estimar adecuadamente la densidad de probabilidad real de los datos [31] (Theis, Oord, Bethge, 2016). Los modelos de difusión en realidad minimizan el ELBO de NLL, deducido sustancialmente de la función de pérdida propuesta por Song, sintetizada en la ecuación (2.1) [7]. Digamos que el proceso de difusión forward sigue un schedule de densidad de probabilidad tal que $q(\mathbf{x}_t|\mathbf{x}_0)$ para inyectar ruido asociado a un tiempo t y que el proceso de denoising aprendido por la red neuronal sigue la forma $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$. Implícitamente, Song propone que el modelo minimice el ELBO de:

$$\log p(\mathbf{x}) = \mathbb{E}_q \log p(\mathbf{x}_0|\mathbf{x}_1) - \sum D_{KL}[q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p(\mathbf{x}_{t-1}|\mathbf{x}_t)] - D_{KL}[q(\mathbf{x}_t|\mathbf{x}_0)||p(\mathbf{x}_t)]$$

maximizando así las divergencias de Kullback-Leibler y por tanto maximizando la log-likelihood, equivalente a minimizar la NLL. Es por eso que parece intuitivo tener en cuenta esta métrica, pues indica cómo de bien está haciendo el modelo su trabajo, para lo que fue diseñado. Esta métrica es la más adecuada por esa razón: es la encargada de medir de forma más cuantitativa cómo de bien se ha entrenado el modelo.

Bits per dimension (BPD)

Esta métrica surge con el propósito de escalar la NLL para que sea interpretable independientemente de las dimensiones de la imagen. La relación entre BPD y NLL es muy sencilla:

$$BPD = \frac{NLL}{\mathcal{D} \log 2}$$

De esta forma:

$$BPD = -\frac{1}{D} \mathbb{E}_{\mathbf{x} \sim P(\mathbf{x})} [\log_2 p(\mathbf{x})]$$

donde $\log_2 p(\mathbf{x})$ puede ser interpretado como la información en bits según la distribución del modelo. Recordemos que $P(\mathbf{x})$ representa la distribución real de los datos y $p(\mathbf{x})$ la estimada por el modelo, de acuerdo con la notación de Theis, Oord, Bethge (2016) [31], que es ligeramente distinta a la que hemos usado en este capítulo hasta ahora. Y entonces, medida sobre un conjunto de prueba de imágenes $\mathbf{x}_{\text{test}} = \{\mathbf{x}_i\}_{i=1}^N$, $\mathbf{x}_i \in \{0, \dots, 255\}^{\mathcal{D}}$:

$$NLL = -\frac{1}{D \times N} \sum_{i=1}^N \log_2 p(\mathbf{x}_i)$$

Fréchet Inception Distance (FID)

La *Fréchet Inception Distance* (FID) fue propuesta por Heusel et al. (2017) [32] como alternativa al Inception Score (que se verá en la siguiente subsección) para superar sus limitaciones a la hora de comparar la distribución de imágenes generadas con la distribución real. A diferencia de NLL y BPD, no requiere acceso explícito a la verosimilitud del modelo: opera directamente sobre las muestras generadas, lo que la hace aplicable a cualquier modelo generativo, incluidos los modelos de difusión.

Extracción de features. El primer paso consiste en obtener una representación vectorial de las imágenes. Para ello se utiliza la red InceptionV3 [33], preentrenada sobre ImageNet, extrayendo la activación de la capa "pool.3" de dimensión $d = 2048$. Sea $\phi : \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^{2048}$ el mapa de features inducido por dicha red. Dado un conjunto de N_r imágenes reales $\{x_i^r\}$ y un conjunto de N_g imágenes generadas $\{x_i^g\}$, se computan los vectores $\phi(x_i^r)$ y $\phi(x_i^g)$ respectivamente.

Ajuste gaussiano. Se asume que ambas distribuciones en el espacio de features pueden aproximarse mediante gaussianas multivariadas:

$$\phi(x^r) \sim \mathcal{N}(\mu_r, \Sigma_r)$$

$$\phi(x^g) \sim \mathcal{N}(\mu_g, \Sigma_g)$$

cuyos parámetros se estiman con los estimadores muestrales de máxima verosimilitud:

$$\hat{\mu}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \phi(x_i^r), \quad \hat{\Sigma}_r = \frac{1}{N_r - 1} \sum_{i=1}^{N_r} (\phi(x_i^r) - \hat{\mu}_r)(\phi(x_i^r) - \hat{\mu}_r)^\top$$

y análogamente para $\hat{\mu}_g$ y $\hat{\Sigma}_g$.

Distancia de Fréchet. La FID se define como la distancia de Fréchet [34] (equivalente a la distancia de Wasserstein-2 bajo la suposición gaussiana) entre las dos distribuciones estimadas:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}), \quad (11)$$

donde $(\Sigma_r \Sigma_g)^{1/2}$ denota la raíz cuadrada matricial del producto $\Sigma_r \Sigma_g$, calculada en la práctica mediante descomposición en valores singulares. La expresión (2.26) tiene una interpretación geométrica clara: el primer término penaliza la distancia entre las *medias* de ambas distribuciones en el espacio de features (calidad media de las muestras), mientras que el segundo término penaliza las diferencias en *covarianza* (diversidad y estructura de las muestras). Así, un modelo que genere imágenes de alta calidad pero poco diversas tendrá un término de covarianza elevado aunque su término de medias sea pequeño.

Un FID igual a cero indicaría que las distribuciones real y generada son idénticas en el espacio de features; cuanto mayor es el FID, peor es la calidad perceptual global del generador. Como referencia, el modelo DDPM [16] reporta un FID de 3,17 en CIFAR-10, mientras que enfoques más recientes como DiT [35] alcanzan valores inferiores a 2,0 en la misma tarea.

Entre sus limitaciones más conocidas, la FID asume gaussianidad en el espacio de features, es sensible al número de muestras utilizadas (se recomienda $N \geq 10\,000$ para estimaciones estables [36]) y hereda los sesgos de InceptionV3 hacia las clases de ImageNet, por lo que su interpretación puede ser menos fiable en dominios muy alejados de la fotografía natural.

Inception Score (IS)

El Inception Score [37] es otra métrica basada en InceptionV3 que cuantifica simultáneamente dos propiedades deseables en un generador de imágenes: la calidad semántica de cada imagen individual y la diversidad del conjunto de imágenes generadas.

Definición. Sea $p(y|x)$ la distribución de probabilidad sobre las C clases de ImageNet predicha por InceptionV3 para una imagen generada x , y sea

$$p(y) = \mathbb{E}_{x \sim p_g}[p(y | x)]$$

la distribución marginal de clases promediada sobre todas las imágenes generadas. El IS se define como:

$$\text{IS} = \exp\left(\mathbb{E}_{x \sim p_g}[D_{\text{KL}}(p(y | x) \| p(y))]\right). \quad (12)$$

Descomposición en entropías de Shannon. Utilizando la descomposición de la divergencia de Kullback-Leibler en términos de entropías de Shannon, $D_{\text{KL}}(p\|q) = H(q) - H(p | q)$, la ecuación (2.28) puede reescribirse de forma más interpretable como:

$$\text{IS} = \exp\left(H(p(y)) - \mathbb{E}_x[H(p(y | x))]\right), \quad (13)$$

donde $H(\cdot)$ denota la entropía de Shannon (en nats si se emplea el logaritmo natural). La ecuación (2.29) hace evidente el doble criterio que optimiza el IS:

- **Calidad:** se desea que la entropía condicional $\mathbb{E}_x[H(p(y | x))]$ sea *baja*, lo que equivale a que el clasificador asigne con alta confianza cada imagen a una clase concreta ($p(y | x) \approx \mathbf{e}_k$ para algún k).

- **Diversidad:** se desea que la entropía marginal $H(p(y))$ sea *alta*, lo que equivale a que el conjunto generado cubra uniformemente todas las clases ($p(y) \approx \text{Uniform}(C)$).

Un generador en colapso de modo (mode collapse) producirá imágenes centradas en pocas clases, reduciendo $H(p(y))$ y penalizando el IS aunque cada imagen individualmente sea nítida.

Estimación práctica. Dado un conjunto de N imágenes generadas, el IS se estima como:

$$\widehat{\text{IS}} = \exp\left(\frac{1}{N} \sum_{i=1}^N \sum_{y=1}^C p(y | x_i) \log \frac{p(y | x_i)}{\hat{p}(y)}\right), \quad \hat{p}(y) = \frac{1}{N} \sum_{i=1}^N p(y | x_i). \quad (14)$$

Es habitual calcular esta estimación en varios subconjuntos de muestras y reportar la media $\pm$ desviación estándar para obtener una estimación más robusta.

Valores de IS representativos en la literatura son: el IS original sobre CIFAR-10 para imágenes reales es $\approx 11,24$ [37]; modelos de difusión como DDPM [16] reportan IS de aproximadamente 9,46 en CIFAR-10. Al contrario que FID, un IS más alto indica mejor calidad y diversidad.

Comparativa de las métricas

La elección de la métrica de evaluación adecuada depende del tipo de modelo, del objetivo de la evaluación y de los recursos computacionales disponibles. La tabla (2) resume las propiedades más relevantes de las cuatro métricas estudiadas.

Tabla 2: Comparativa de métricas de calidad para modelos generativos de imágenes.

Métrica	NLL	BPD	FID	IS
Requiere verosimilitud	Sí	Sí	No	No
Computable exacto	No (ELBO)	No (ELBO)	Sí	Sí
Detecta diversidad	Sí	Sí	Sí (Σ)	Parcial
Depende de red externa	No	No	InceptionV3	InceptionV3
Compara con datos reales	Sí	Sí	Sí	No
Normalizada por tamaño	No	Sí	No	No
Mejor valor	Mín.	Mín.	Mín.	Máx.

NLL y BPD. Estas métricas son las más directamente vinculadas a la función objetivo de entrenamiento de los modelos de difusión, pues estos minimizan el ELBO de la NLL [16]. Su principal ventaja es que cuantifican de forma precisa cuán bien el modelo ha aprendido la densidad de probabilidad real. Sin embargo, Theis et al. [31] demostraron formalmente que una NLL baja no garantiza que las muestras generadas sean perceptualmente buenas: un modelo puede asignar alta verosimilitud a imágenes ruidosas o borrosas mientras que genera muestras de baja calidad visual. Por ello, la NLL y BPD son más adecuadas para evaluar la capacidad de modelado de la distribución, no la calidad visual de las muestras.

FID. La FID es actualmente la métrica más utilizada en la literatura de generación de imágenes, tanto para GANs [32] como para modelos de difusión, ya que captura de forma conjunta la calidad y la diversidad de las muestras en un único escalar. Su ventaja principal

frente al IS es que compara las imágenes generadas con imágenes reales, haciendo la evaluación más fiel a la distribución objetivo. No obstante, su dependencia de InceptionV3 introduce un sesgo hacia las clases de ImageNet y su suposición de gaussianidad en el espacio de features puede no reflejar fielmente la geometría real de las distribuciones [36].

IS. El IS es útil como métrica complementaria de diversidad semántica, especialmente cuando no se dispone de un conjunto de imágenes reales de referencia. Sin embargo, su principal limitación es que no compara directamente con la distribución real: un generador que memorice el conjunto de entrenamiento podría obtener un IS muy alto con FID bajo o nulo. Además, el IS es especialmente sensible al dominio: en conjuntos de datos muy distintos de ImageNet, las predicciones de InceptionV3 pierden significado semántico y el IS deja de ser un indicador fiable [38].

Recomendación práctica. En el contexto de los modelos de difusión, la práctica habitual en la literatura consiste en reportar simultáneamente FID y NLL/BPD: la primera para evaluar la calidad perceptual de las muestras y la segunda para evaluar la bondad del ajuste probabilístico del modelo. El IS se reporta con menor frecuencia, pero puede resultar informativo en conjuntos de datos con clases bien definidas (p. ej., ImageNet o CIFAR-10) para diagnosticar fenómenos de colapso de modo. Métricas más recientes como CLIP-FID [39] o la *Precision and Recall* [40] permiten desacoplar la evaluación de calidad y diversidad, constituyendo una alternativa más informativa a IS y una extensión natural de FID.